Customer Churn Early-Warning Dashboard

Final Project Report

BA6003 Database & Programming

14 October 2026

Group Number: 6

Group Members

Li Zhou

Zhou Siyan

Yao Bowen

Abstract

This project uses a telecom customer dataset to examine whether churn risk can be identified early enough to help a retention team decide whom to contact. We cleaned and stored the data in SQLite, explored churn patterns with SQL, and compared two classification models. We selected logistic regression for the dashboard because its predictions are relatively easy to explain. In our test results, the High and Critical groups contained about 22% of customers and accounted for 54% of observed churn. We also built a call-script feature and a retention-cost simulator. The estimated benefit depends on assumptions about contact costs and successful retention; it is not a result measured from a live campaign.

Contents

1 Business problem and users 1

2 Data and database design 1

3 What the SQL analysis shows 2

4 Churn model 4

5 Dashboard 5

6 GenAI assistant and verification 6

7 Business impact 7

8 Limitations and next steps 8

9 Reproducibility 9

1 Business problem and users

For a subscription business, it is useful to identify customers who may leave before they cancel. In this dataset, 1,869 of the 7,043 customers had churned, giving an overall churn rate of 26.5%. Their monthly charges add up to USD 139,131. This figure shows the monthly charges associated with customers who left; it should not be treated as a guaranteed loss in a future month. Our project therefore focuses on finding churn patterns and using them to help prioritise customer contact.

We developed a Customer Churn Early-Warning Dashboard to bring these steps together. It ranks customers by predicted risk, displays the main factors behind a score, and provides a draft call script for staff to review. A separate simulator estimates the possible cost and return of a retention campaign. The intended users are customer-success teams, call-centre agents and marketing staff. Telecom data is used as the case study here. A bank or insurer could use a similar workflow, but its data, risk thresholds and retention offers would need to be assessed separately.

The table maps the seven discussion prompts of the project brief to this report.

Brief prompt	Our answer	Section
1. Who is the target user?	Customer-success teams, call-centre agents, marketing staff	1
2. What decision or insight do we support?	Which at-risk customers to contact first, with which offer, and whether the campaign pays back	1, 7
3. What data do we need?	Telco Customer Churn: demographics, services, contract, billing and a churn label	2
4. What SQL queries are needed?	Joins, filters, GROUP BY, HAVING, CTE and window functions on a three-table schema (11 named queries)	2, 3
5. What can the user interact with?	Filters, call list, customer drill-down, script generator, ROI sliders	5
6. What does the AI assistant generate or explain?	A personalised call script and plain-language reasons for each flag	6
7. How do we stay simple but complete, with AI outputs verified?	One end-to-end workflow; every AI output is checked by rules and tests	6, 8

2 Data and database design

The analysis uses the Telco Customer Churn dataset hosted on Kaggle. The file WA_Fn-UseC_-Telco-Customer-Churn.csv contains 7,043 customer records and 21 columns, including customer characteristics, subscribed services, contract and billing information, and a Yes/No churn label. Before loading it into the database, we checked duplicate IDs, missing values and column types. The main cleaning decisions are summarised below.

Check	Finding	Action
Duplicate customer IDs	None; 7,043 unique	None needed
Blank TotalCharges	11 rows, all with tenure = 0 (new customers not yet billed)	Stored as NULL, not imputed
Data type	TotalCharges arrives as text	Converted to numeric
Churn label	Yes/No text	Mapped to 1/0
Class balance	26.5% churned	Stratified train/test split
Load reconciliation	Every table must hold 7,043 rows	Asserted in the ETL script

We used SQLite for the database. The design has four tables and one analysis view, separating customer details, services, billing information and model scores. Primary and foreign keys help keep the records linked correctly, while CHECK constraints restrict selected fields to valid values. We also added indexes to columns used frequently in filters. These measures help reduce data-quality problems, but do not replace checks during data loading [1].

Table	One row per	Main columns
customers	customer	customer_id (key), gender, senior_citizen, partner, dependents, tenure_months, churn
services	customer	phone and internet service plus six add-ons (security, backup, protection, tech support, streaming)
billing	customer	contract, paperless_billing, payment_method, monthly_charges, total_charges
churn_scores	customer	churn_prob, risk_tier, top_drivers, scored_at (written back by the Python model)
v_customer_360 (view)	customer	all of the above plus a tenure band; the single source for queries and the dashboard

3 What the SQL analysis shows

The SQL results suggest that churn is not evenly distributed across customer groups. Contract type shows the clearest difference, while tenure, internet service and payment method also vary noticeably. The figures below are taken from the queries in sql/02_queries.sql. These comparisons describe patterns in this dataset; on their own, they do not establish that any feature causes a customer to leave.

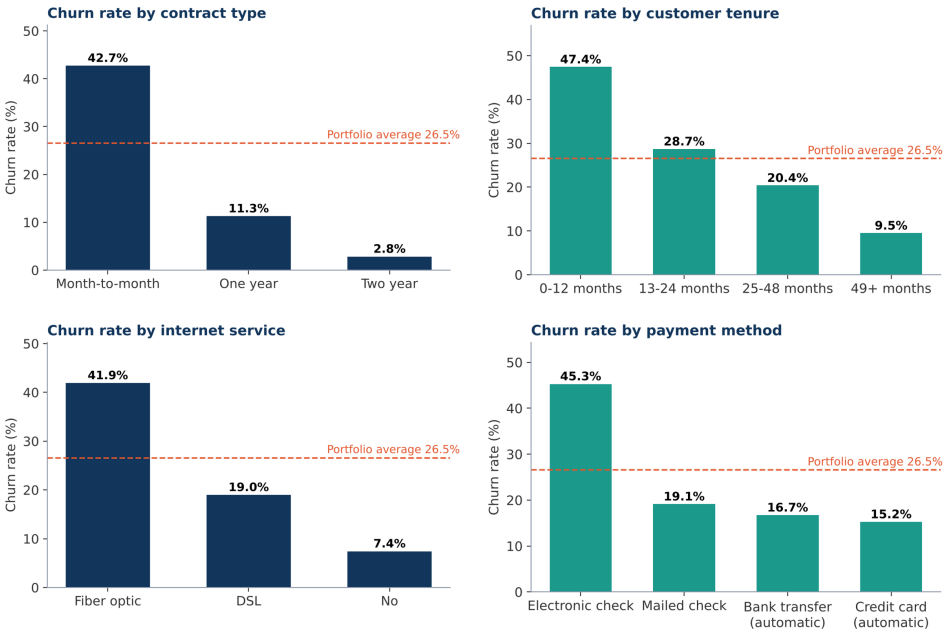


Figure 1: Churn rate by contract, tenure, internet service and payment method (dashed line = 26.5% portfolio average)

- **Contract:** month-to-month customers churn at 42.7%, against 11.3% on one-year and 2.8% on two-year contracts. They are 88.6% of all churners (1,655 of 1,869).
- **Tenure:** 47.4% of customers in their first 12 months churn, against 9.5% after 48 months.

- **Internet service:** fiber optic customers churn at 41.9% against 19.0% for DSL, and they also pay the most (USD 91.50 a month on average).
- **Payment method:** electronic-check payers churn at 45.3%; the other three methods sit between 15.2% and 19.1%.
- **Protective add-ons:** among internet customers, those with both Online Security and Tech Support churn at 9.0%, against 49.0% for those with neither.

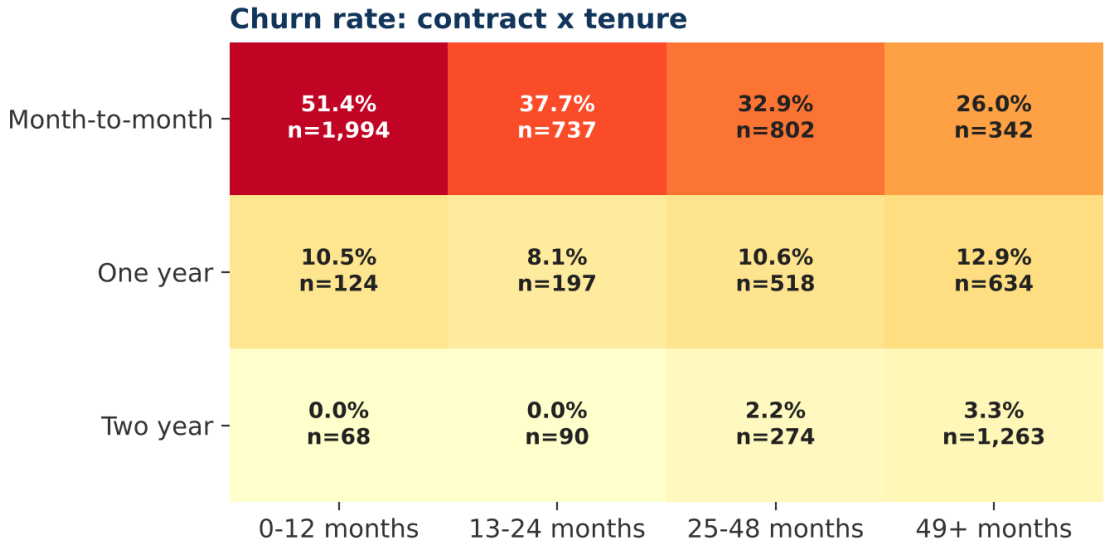


Figure 2: Churn rate by contract and tenure band (n = customers in each cell)

The segment with the highest observed churn rate consists of month-to-month customers using fiber optic service who are in their first year. It contains 916 customers, has a churn rate of 70.2%, and accounts for USD 75,183 in monthly charges. The two-year-contract groups show 0.0% churn in the displayed cells, but their sample sizes are only 68 and 90. We therefore interpret these figures cautiously rather than assuming churn is impossible for this group.

The table lists the eleven queries and the technique each one demonstrates.

Query	SQL technique	Business question
portfolio_kpis	Aggregates	What is the headline churn rate and revenue lost?
churn_by_contract	GROUP BY, scalar subquery	Which contract carries the churn, and what share of the total?
churn_by_tenure_band	CASE band in view, GROUP BY	When do customers leave?
churn_by_internet_service, churn_by_payment_method	JOIN via view, GROUP BY	Which products and payment methods carry risk?
churn_by_support_addons	WHERE, CASE, GROUP BY	Do protective add-ons reduce churn?
high_risk_segments	GROUP BY, HAVING (at least 100 customers)	Which segments should we target first?
churn_vs_portfolio_average	CTE, RANK() window function	How far is each segment from the average?
risk_tier_summary	JOIN to model scores	Do the model's tiers match what happened?

Query	SQL technique	Business question
top_at_risk_customers	JOIN, WHERE, ORDER BY	Who should agents call first (active customers only)?
active_base_by_tier	Filtered GROUP BY	How much revenue sits in each tier of the live base?

4 Churn model

On the held-out test set, logistic regression achieved an ROC-AUC of 0.845. The observed churn rate also increased across the four risk tiers, suggesting that the ranking separates groups with different levels of risk. We considered gradient boosting as a second model before deciding which one to use in the dashboard.

The model uses 17 inputs (tenure, monthly charges and 15 categorical fields on services, contract and billing). We excluded gender, which has no business use, and TotalCharges, which is roughly tenure times monthly charge and made coefficients misleading; dropping it moved ROC-AUC by less than 0.002. We held out 25% of customers (1,761) for testing and compared the model with a gradient-boosting challenger.

Hold-out metric	Logistic regression	Gradient boosting
ROC-AUC	0.845	0.844
PR-AUC	0.635	0.653
Brier score (lower is better)	0.137	0.136
Precision at 0.5	64.7%	66.7%
Recall at 0.5	54.8%	48.4%
Top-decile lift (baseline churn 26.5%)	2.76x	2.89x

The test metrics are close. Gradient boosting has a slightly higher PR-AUC, while logistic regression has higher recall at the 0.5 threshold. We kept logistic regression for the dashboard because its coefficients are easier to inspect and explain to staff [3]. The scores written to the database are generated using five-fold out-of-fold predictions, with an ROC-AUC of 0.843. This reduces the risk of reporting a score from a model trained on that same customer[2].

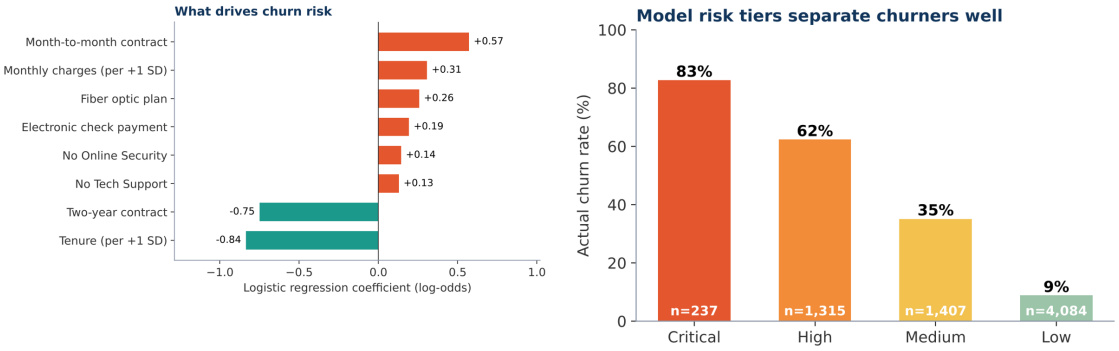


Figure 3: Left, model drivers (log-odds; positive raises risk). Right, actual churn rate by risk tier (n = customers per tier)

Risk tiers cut the predicted probability at 0.25, 0.50 and 0.75. Actual churn was 8.8% in Low (4,084 customers), 35.1% in Medium (1,407), 62.3% in High (1,315) and 82.7% in Critical (237). Flagging High and Critical flags 1,552 customers (22.0% of the base)[3], captures 54.3% of all churners and is right 65.4% of the time [2].

The model results broadly agree with the patterns found in the SQL analysis. Month-to-month contracts, higher monthly charges, fiber optic service, electronic-check payments, and the absence of Online Security or Tech Support are associated with higher predicted risk in this model. Longer tenure and two-year contracts are associated with lower risk. These coefficients describe associations within the fitted model, not causal effects. For each customer, the dashboard displays the three main behavioural factors contributing to the score. Demographic variables such as senior-citizen status, partner status and dependents are not displayed as explanations or used in the generated scripts [5].

5 Dashboard

We built the dashboard in Streamlit and connected its views to the SQLite database through SQL queries. It has four main areas: the overall customer base, the call list, individual customer details, and the retention ROI simulation. By default, it excludes customers who have already left. This leaves 5,174 active customers, including 537 in the High or Critical tiers. Their monthly charges total USD 67,947; this is the amount associated with these customers, not a guaranteed revenue loss [4].

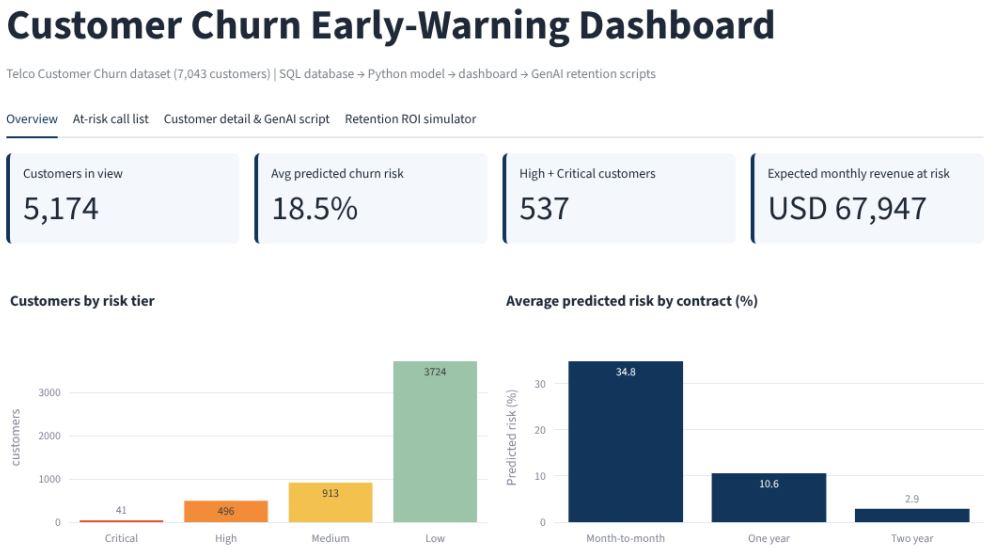


Figure 4: Overview tab with KPI tiles and risk-tier charts (sidebar filters not shown)

Tab	What the user sees	What the user can do
Overview	KPI tiles, customers by risk tier, average risk by contract, highest-risk segments	Filter by contract, internet service and tenure band; switch between active customers and all customers (back-test)
At-risk call list	Customers ranked by expected monthly loss (churn probability x monthly charge) with the top three reasons	Choose risk tiers; download the list as CSV
Customer detail and GenAI script	Churn probability, tier, contract facts and why the customer was flagged	Pick a customer, generate an editable call script and see its verification result

Tab	What the user sees	What the user can do
Retention ROI simulator	Customers contacted, expected churners reached, expected saves, net benefit, ROI and a net-benefit curve	Move the contact threshold, cost per contact, save rate, months of revenue kept and discount cost

We tested the app with an automated headless run that opens every tab, toggles the filters and clicks the script button; it finished with no errors [5].

6 GenAI assistant and verification

The script feature is intended to help agents prepare for a call, not to make retention decisions automatically. Predefined rules select the offer, while the language model receives customer facts and offers that have already been approved. A set of checks looks for unsupported figures and unsuitable wording before the draft is shown. Staff still need to review the result and can edit it before use.

1. **Offer selection (rules, not AI).** Month-to-month customers get a 10% discount for 12 months on a one-year contract; electronic-check payers get a waived paper-bill fee plus a one-off USD 5 credit for automatic payment; internet customers without Tech Support or Online Security get 3 months free; otherwise a free plan review. At most two offers are used.

2. **Prompt with approved facts only.** The model receives tenure, contract, internet plan, current monthly bill and the approved offers, plus rules: 80-130 words, no mention of churn, risk or scores, no personal traits, no invented prices or deadlines, and a closing question.

3. **Generation.** The script is written by Claude through the Anthropic API when a key is set, and by a deterministic template when offline, so the workflow always runs.

4. **Verification and human review.** The dashboard shows "Verification passed" or "Needs human review" with the reasons, and the agent can edit the text before use. Names are placeholders for the agent to fill in.

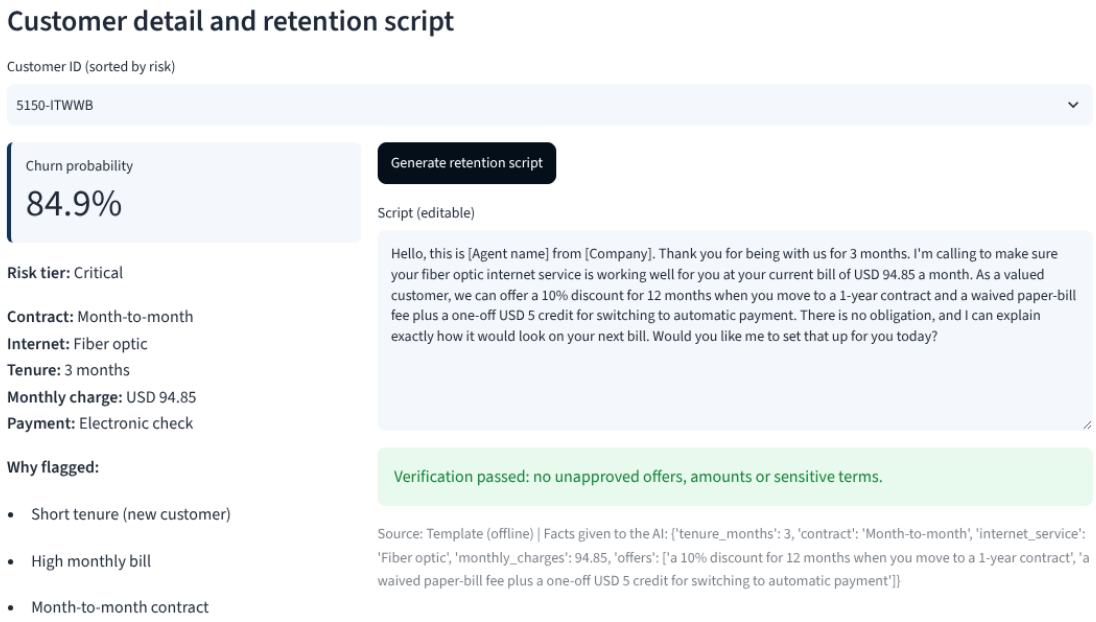


Figure 5: Customer detail tab with the reasons for the flag, an editable script and the verification result

AI risk	Control in code	Evidence
Invented discounts, prices or durations	Every USD amount, percentage and number of months must match the customer's real bill, real tenure or the approved offer list	Test scripts with a 50% discount or an unapproved price are rejected
Sensitive or inappropriate wording	Forbidden-term check (churn, probability, scores, age, gender, family status)	A script mentioning a "senior customer" is rejected
Off-brief output	Length 60-150 words, must mention an approved offer, must end with a question	A script with no call to action is rejected
LLM output fails the checks	The text is replaced by the template and labelled as such	Failure path covered in code
Our own mistakes	Seven automated tests (verifier and database integrity)	All 7 pass; all 7,043 template scripts pass the verifier

The model's numbers are verified separately: row counts are reconciled at every load step, the score table must cover every customer with probabilities between 0 and 1, and every figure in this report was taken from the pipeline's output files (summary.json and the sql_results folder) and checked against them.

7 Business impact

Under the assumptions used in the simulator, starting with the 537 active customers in the High and Critical tiers gives an estimated net benefit of USD 84,372 and an ROI of 596%. This is a scenario estimate, not a result observed from an actual retention campaign. We restrict the calculation to active customers because the model is intended to support prevention of future churn, rather than recovery of customers who have already left.

Assumptions (all editable in the dashboard): each contact costs USD 8; 30% of contacted would-be churners are saved; a saved customer keeps 12 months of revenue; the retention discount costs 10% of the revenue kept. For each customer, expected benefit = churn probability x save rate x monthly charge x 12 months; a customer is worth contacting when that exceeds the contact cost plus the discount.

Risk tier	Active customers	Expected saves	Cost (USD)	Net benefit (USD)	ROI
Critical	41	9.6	1,352	8,890	657%
High	496	90.9	12,796	75,482	590%
Medium	913	98.6	15,582	67,194	431%
Low	3,724	88.4	36,123	27,191	75%

Based on this simulation, the team could contact Critical and High customers first, then consider the Medium tier if agent capacity allows. Low-risk customers could receive lower-cost messages instead. This is a starting point rather than a fixed rule: the order should be reviewed if contact costs, retention success rates or the value of saved customers differ from the assumptions used here.

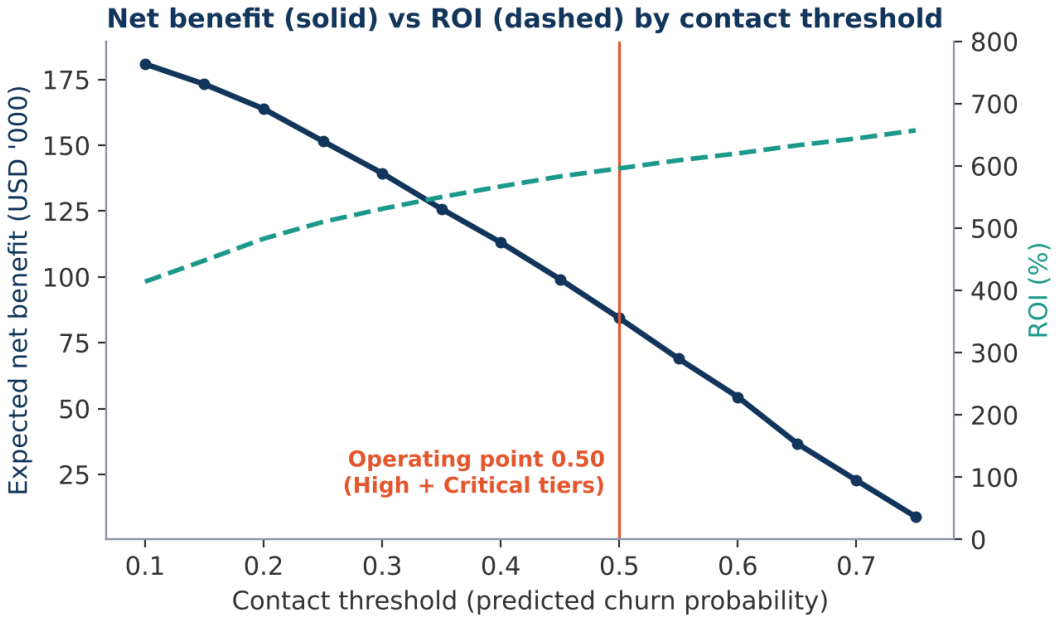


Figure 6: Expected net benefit (solid) and ROI (dashed) as the contact threshold moves; raising the threshold lifts ROI but reduces total value

The result depends most on the save rate, which we have assumed, not measured. The campaign stays profitable even if only one in ten contacted churners is saved.

Save rate	Expected saves	Net benefit (USD)	ROI
10%	33.5	25,260	333%
20%	67.0	54,816	505%
30%	100.5	84,372	596%

8 Limitations and next steps

The project focuses on one use case and one dataset so that we could build and test a complete workflow, from data preparation to the dashboard. There are still several limits to keep in mind:

- **Snapshot data.** The dataset has no dates, so scores show relative risk and the ROI tab is a decision-support simulation, not a forecast. Active customers have not left yet, and we treat their probabilities as forward-looking risk.
- **Assumed economics.** Contact cost, save rate, months of revenue kept and discount cost are assumptions, not measurements.
- **Industry proxy.** Telecom behaviour is a stand-in for banks and insurers; the model and offers must be retrained and recalibrated on the institution's own data.
- **GenAI scope.** The live LLM call is optional and was not exercised in our test run, which had no API key; the template path, the verifier and the fallback logic were tested. The verifier checks facts and wording but cannot judge tone, so agents read and edit every script.
- **Fairness.** Gender is excluded and demographic traits are never shown or used in scripts, but contract and tenure patterns can still correlate with protected groups, so outcomes should be monitored by segment.

Next steps:

1. Collect time-stamped data so the model predicts churn within the next 30 days.
2. A/B test the scripts against a control group to measure the real save rate.
3. Add an uplift model to target customers who are persuadable, not only at risk.
4. Connect the database to the CRM so the call list refreshes nightly.

9 Reproducibility

The project can be run from the repository using the commands below. The README explains the setup process, key design choices, verification steps and known limitations. These instructions assume that the required packages are installed and the data file is available in the expected location.

```
pip install -r requirements.txt
```

```
python run_pipeline.py                    data → SQLite → model → analysis
```

```
streamlit run app/dashboard.py            open the dashboard
```

```
python -m pytest -q                       7 tests
```

Path	Purpose
sql/01_schema.sql, sql/02_queries.sql	Tables, constraints, analysis view and the eleven named queries
src/etl.py	Clean, validate and load the CSV into SQLite
src/model.py	Train, evaluate and score the model; write scores and reasons back to SQL
src/roi.py	Retention campaign ROI logic, shared by the dashboard and this report
src/genai.py	Prompt, script generation (LLM or template) and the verification layer
src/analysis.py	Run the queries; produce figures and summary.json
app/dashboard.py	Streamlit dashboard
tests/	Automated tests
outputs/	metrics.json, summary.json, SQL result CSVs and figures

Each module is short and commented with the reasoning behind its design choices, so the team can walk through its own code line by line.

References

[1] Blastchar. *Telco Customer Churn Dataset*. Kaggle. Available at: https://www.kaggle.com/datasets/blastchar/telco-customer-churn.

[2] Fawcett, T. (2006). An introduction to ROC analysis. *Pattern Recognition Letters, 27*(8), 861–874. https://doi.org/10.1016/j.patrec.2005.10.010.

[3] Kuhn, M., & Johnson, K. (2013). *Applied Predictive Modeling*. Springer. https://doi.org/10.1007/978-1-4614-6849-3.

[4] Streamlit. *Streamlit Documentation: Multipage Apps*. Available at: https://docs.streamlit.io/develop/concepts/multipage-apps/overview.

[5] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830. Available at: https://jmlr.org/papers/v12/pedregosa11a.html.